

Aganitha Take-Home Python Exercise Report – 2025

Candidate

- . Name: Likhitha Sompalli
- . Email: likithalikki187@gmail.com

Problem Understanding

The objective is to develop a command-line Python tool that queries PubMed using their API, filters papers that have at least one **non-academic author** (typically affiliated with a pharmaceutical or biotech company), and exports the results into a CSV file.

Approach & Methodology

- Utilized PubMed's **E-utilities API** to perform both search and fetch operations.
- Parsed the returned XML data using Python's `xml.etree.ElementTree` to extract:
 - PubMed ID
 - Title
 - Publication Date

- Author Affiliations
- Author Emails
- Applied **simple heuristics** to detect non-academic affiliations by checking the absence of keywords like "university", "institute", "college", etc.
- Developed the tool as a **modular package** with a clear separation between fetching, filtering, and utility logic.
- Command-line interface was built using argparse with support for query input, debug mode, and output file path.
- **Poetry** was used for package management and dependency setup.

Code Structure

- pubmed_papers/
 - fetcher.py: Handles API requests and parses XML data.
 - filters.py: Contains logic for identifying non-academic affiliations.
 - utils.py: Email extraction helper.
- cli/get_papers_list.py: Main CLI application.
- tests/test_fetcher.py: Contains unit test for the fetch logic.
- pyproject.toml: Poetry configuration file.
- README.md: Documentation and usage instructions

Sample Output

For the query "cancer AND 2022[dp]", the output CSV contains the following fields:

- **PubmedID**
- **Title**
- **Publication Date**
- **Non-academic Author(s)**

- **Company Affiliation(s)**
- **Corresponding Author Email**

Only papers with at least one non-academic author are included as per the requirement

Challenges & Learnings

- Parsing and interpreting real-world XML data from PubMed was challenging and required careful traversal of the tree structure.
- Designing heuristics that are effective and efficient for real affiliation strings.
- Learned how to build a professional Python CLI tool using **Poetry**, and how to make it modular and testable.

Conclusion

The CLI tool fulfills all specified requirements. It efficiently queries PubMed, identifies papers with authors from pharmaceutical or biotech companies using heuristics, and saves structured results in CSV format. The codebase is modular, testable, and packaged professionally.

Future enhancements could include:

- NLP-based classification of affiliations
- Extended email verification
- Automated scheduling and reporting